CS4475 Fall 2026
Aden Ramirez
aeramirez17

Lab Report
Submitted on October 14, 2026

PA2: Priority-based Scheduler for xv6

Repository: https://github.com/adeelone/xv6 (branch `pa2`, which starts from my `pa1-syscalls` branch so it keeps `uptime`, `time`, `wait2` and `cputime`).

New files: `user/ps.c`, `user/pexec.c`, `user/testprio.c`, `user/schedtest.c`, `user/spin.c`

Modified files: `Makefile`, `kernel/param.h`, `kernel/pstat.h`, `kernel/proc.h`, `kernel/proc.c`, `kernel/defs.h`, `kernel/syscall.h`, `kernel/syscall.c`, `kernel/sysproc.c`, `user/user.h`, `user/usys.pl`, `user/sh.c`. `CPUS` was already set to 1 in the `Makefile` from PA1.

Task 1. (15 pts) Implement the `getprocs()` system call and test it using the provided `ps` command.

`getprocs(struct pstat *)` is a new system call (number 24). It walks the kernel process table and copies one `struct pstat` out to the user array for every process that is in use, then returns how many entries it filled in.

The real work is in `kgetprocs()` in `kernel/proc.c`. It acquires `wait_lock` first (because `p->parent` is protected by `wait_lock`, and `kwait()` takes `wait_lock` before any `p->lock`, so I used the same order), then loops over `proc[NPROC]`. For each slot it takes `p->lock`, skips `UNUSED` and `USED` slots (`USED` is a half-built process with no name yet), and copies `pid`, `state`, `sz`, the parent's pid (0 for `init`, which has no parent) and name into a local `struct pstat`. It releases `p->lock` and then calls `copyout()` to write the entry to `addr + n * sizeof(struct pstat)` in the caller's address space. `sys_getprocs()` in `kernel/sysproc.c` just reads the user pointer with `argaddr()` and calls `kgetprocs()`.

Files changed for Task 1:

- `kernel/pstat.h`: added `struct pstat`. The provided `ps.c` uses `enum procstate`, which was only defined in `kernel/proc.h`, and user programs cannot include `proc.h` because it pulls in spinlock and other kernel types. I moved the enum into `pstat.h` (with an include guard) and made `proc.h` `#include "pstat.h"`, so the kernel and `ps` share one definition.
- `kernel/proc.c`: `kgetprocs()`
- `kernel/defs.h`: its prototype
- `kernel/syscall.h`: `SYS_getprocs 24`
- `kernel/syscall.c`: extern declaration and table entry
- `kernel/sysproc.c`: `sys_getprocs()`

- `user/user.h`: `struct pstat;` and `int getprocs(struct pstat*);`
- `user/usys.pl`: `entry("getprocs")`
- `Makefile`: `$U/_ps` added to `UPROGS`

I also fixed one bug in the provided `ps.c`: it printed size with "`%l`", which xv6's `printf` does not support (it only understands `%ld`, `%lu`, `%lx`), so the size column came out wrong. I changed it to `%ld`. I also added `UNUSED` and `USED` to the `states[]` array so an unexpected state can never index a null string.

Figure 1 shows `ps` on a freshly booted kernel. It lists `init` (`pid` 1), the shell (`pid` 2) and `ps` itself (`pid` 3), each with the right parent (`ppid` 0, 1 and 2). The columns after size (`age`, `prio`, `eprio`, `cputime`) were added in Tasks 2, 3 and 5 and PA1, so this screenshot comes from the final `ps`; when I finished Task 1 it printed only `pid`, `state`, `size`, `ppid` and `name`. `ps` sees itself as running because `getprocs()` is called while `ps` is on the CPU, and the other two are sleeping (`init` in `wait()`, `sh` in `read()`).

```
init: starting sh
$ ps
pid     state           size    age     prio    eprio   cputime ppid    name
1       sleeping        16384   0       0       0       0       0       init
2       sleeping        20480   0       0       0       1       1       sh
3       running         16384   0       0       0       0       2       ps
$
```

Figure 1. ps (getprocs) listing every live process

Task 2. (15 pts) Modify the provided `ps` command to print the priority of each process.

I added `int priority` to `struct proc`. `allocproc()` sets it to 0, `userinit()` explicitly sets `init` to 0, and `kfork()` copies the parent's value into the child (`np->priority = p->priority`), so a child always inherits its parent's priority.

`getpriority()` (syscall 25) returns `myproc()->priority`. `setpriority(int)` (syscall 26) reads the argument with `argint()`, rejects anything outside 0..`MAXPRIORITY` (49) by returning -1, and otherwise stores the new value under `p->lock` and returns the old priority. The upper limit is 49 because base priorities are 0 to 49 and only aging can push the effective priority up to 99. Both calls were registered the same way as `getprocs` (`syscall.h`, `syscall.c`, `sysproc.c`, `user.h`, `usys.pl`).

For `getprocs()` I added a priority field to `struct pstat` and one line in `kgetprocs()`: `ps.priority = p->priority`. `ps.c` prints it in the new `prio` column.

My test program `user/testprio.c` checks the starting priority, sets it to 20, tries two invalid values (60 and -1), forks a child that prints the priority it inherited and then lowers itself to 7, and finally runs `ps`. Figure 2 shows the output: the old priority is returned correctly, both invalid values return -1 and leave the priority at 20, the child inherits 20, and `ps` shows the parent and `ps` (also a child of `testprio`) at 20 and the child at 7.

```
3       running         16384   0       0       0       0       2       ps
$ testprio
testprio: starting priority = 0
testprio: setpriority(20) returned 0 (old priority)
testprio: priority now = 20
testprio: setpriority(60) returned -1 (should be -1)
testprio: setpriority(-1) returned -1 (should be -1)
testprio: priority still = 20
testprio child: inherited priority = 20
testprio child: set own priority to 7
pid     state           size    age     prio    eprio   cputime ppid    name
1       sleeping        16384   0       0       0       0       0       init
2       sleeping        20480   0       0       0       1       1       sh
4       sleeping        16384   0       20      20      1       2       testprio
5       runnable        16384   0       7       7       0       4       testprio
6       running         16384   0       20      20      0       4       ps
$
```

Figure 2. testprio: getpriority/setpriority, range check and inheritance

Task 3. (15 pts) Add a `readytime` field to `struct proc`, initialize it correctly, and modify `ps` to print a process's age.

I added `uint readytime` to `struct proc`. It holds the value of the global `ticks` counter at the moment the process last became `RUNNABLE`. I set it at every place in `kernel/proc.c` where a state changes to `RUNNABLE`: `userinit()` (`init`), `kfork()` (new child), `yield()` (timer preemption), `wakeup()` (end of a sleep) and `kkill()` (a killed sleeper is made runnable). In `wakeup()` I read `ticks` without acquiring `tickslock`, because `clockintr()` calls `wakeup(&ticks)` while it already holds `tickslock` and taking it again would deadlock; reading a single aligned `uint` is safe enough here.

The age of a process is how long it has been waiting in the ready queue: age = current time - `readytime`, where current time is `uptime()` in ticks. I added `readytime` to `struct pstat`, and `ps.c` calls `uptime()` once and prints `now - readytime` for `RUNNABLE` processes and 0 for every other state, since a sleeping or running process is not waiting for the CPU.

Figure 3 shows age growing. It was taken on the priority scheduler with aging turned off (`AGING=0`, so `eprio` equals `prio`). After `setpriority 20` the shell is at priority 20, and the background `spin` it starts inherits 20, so the two of them keep the CPU and `init` (priority 0) is left waiting in the ready queue. When `ps` runs, `init` is `RUNNABLE` with an age of 16 ticks. Every process that is sleeping or running shows age 0, as intended. After the `spin` finishes, the second `ps` shows nothing waiting.

```
$ ps
pid     state           size    age     prio    eprio   cputime ppid    name
1       runnable        16384   16      0       0       1       0       init
2       sleeping        20480   0       20      20      0       1       sh
20      running         16384   0       20      20      0       2       ps
19      runnable        16384   0       20      20      16      1       spin
$ spin pid 19 (prio 20) done: 20 ticks
```

Figure 3. ps showing the age of a RUNNABLE process (priority scheduler, AGING=0)

Task 4. (15 pts) Implement a priority-based scheduler.

Files and functions changed

- `kernel/param.h`: added constants to choose the scheduler at compile time (Figure 4). `SCHEDPOLICY` picks `SCHED_RR` or `SCHED_PRIORITY`, and `AGING` turns aging on or off. Both use `#ifndef` so they can also be overridden without editing the file.
- `Makefile`: added `SCHED` and `AGING` variables that pass `-D` flags, so `make clean; make qemu SCHED=SCHED_RR` builds the original round-robin kernel and `make clean; make qemu AGING=0` builds the priority scheduler without aging.

```
#define SCHED_RR       0   // original xv6 round-robin
#define SCHED_PRIORITY 1   // highest effective priority runs first
#ifndef SCHEDPOLICY
#define SCHEDPOLICY SCHED_PRIORITY
#endif
#ifndef AGING
#define AGING 1
#endif
#define MAXPRIORITY    49
#define MAXEFFPRIORITY 99
#define AGINGTICKS     1   // +1 effective priority per tick waited
```

Figure 4. Scheduler selection constants in kernel/param.h

In `kernel/proc.c`, I kept the original round-robin scheduler under `#if SCHEDPOLICY == SCHED_RR` and added the priority scheduler under `#elif SCHEDPOLICY == SCHED_PRIORITY`. An invalid setting produces a compile-time error.

The priority scheduler scans the process table and picks the `RUNNABLE` process with the highest effective priority. If two processes have the same priority, it chooses the one with the earlier `readytime`. If their timestamps are also equal, the first one found wins, which the assignment allows. It then takes the selected process's lock again, checks that it is still `RUNNABLE`, and switches to it. With one CPU and interrupts disabled, its state cannot change between those steps. If nothing is runnable, the scheduler executes `wfi`. Figure 5 shows this code.

```
    // Pass 1: find the best candidate.
    best = 0; besteff = -1; bestready = 0;
    for (p = proc; p < &proc[NPROC]; p++) {
      acquire(&p->lock);
      if (p->state == RUNNABLE) {
        eff = effpriority(p);
        if (eff > besteff || (eff == besteff && p->readytime < bestready)) {
          best = p; besteff = eff; bestready = p->readytime;
        }
      }
      release(&p->lock);
    }
    if (best == 0) { asm volatile("wfi"); continue; }
    // Pass 2: lock the winner, make sure it is still RUNNABLE, run it.
    p = best;
    acquire(&p->lock);
    if (p->state == RUNNABLE) {
      p->state = RUNNING;
      c->proc = p;
      swtch(&c->context, &p->context);
      mycpu()->intena = 0;
      c->proc = 0;
    }
    release(&p->lock);
```

Figure 5. Core of the priority scheduler in kernel/proc.c

- `kernel/proc.c`, `effpriority()`: returns the priority the scheduler compares. In Task 4 (`AGING=0`) it is just `p->priority`; Task 5 adds aging here.
- `kernel/proc.h`, `allocproc()`, `kfork()`, `userinit()` and the `readytime` changes are from Tasks 2 and 3.
- `user/pexec.c` (provided) runs a command at a given priority; I added a check that `setpriority()` succeeded and an error message if `exec` fails.

Test results

I used two test programs. `user/spin.c` runs a CPU-bound loop and prints how long it took. One unit of work takes roughly one tick, although this varied between runs.

`user/schedtest.c` takes an amount of work and a list of priorities, then forks one child for each priority. The parent raises its priority to 49 while creating the children. Each child tries to wait until the same release tick, T0, before starting its work. This gets them started at about the same time, but it is not a strict barrier: the program does not check that every child reached `pause()` before T0, and the separate `uptime()` calls leave a small timing window.

Each child records its response time and turnaround time relative to T0 and sends the results through a pipe. The parent prints the results in finishing order and calculates the averages.

Figure 6 shows `schedtest 20 10 20 30 40` on the priority scheduler without aging. The jobs finish strictly in priority order, even though they were created in the opposite order. The priority 40 job starts at tick 0 and finishes at 27, the priority 30 job starts at 27 (right when 40 finishes), the priority 20 job at 54, and the priority 10 job does not get the CPU at all until tick 80. The average response time is 40.2 ticks and the average turnaround is 66.7 ticks.

```
init: starting sh
$ schedtest 20 10 20 30 40
pid     prio    response        turnaround   (ticks, in finishing order)
7       40      0               27
6       30      27              54
5       20      54              80
4       10      80              106
average response   = 40.2 ticks
average turnaround = 66.7 ticks
$
```

Figure 6. schedtest on the priority scheduler (AGING=0)

Figure 7 uses `pexec` the way the assignment sample does: one `spin` at priority 0 and one at priority 5 in the background, then `ps`. The first `spin` was alone on the CPU while I typed the second command, so it had already used 32 ticks when the first `ps` ran (`cputime` 32). Once the priority 5 job started, the shell, which is also at priority 0, could not run, so `ps` did not execute until the priority 5 job finished (51 ticks). That is why the first `ps` table only appears after the "`spin pid 12 (prio 5) done`" line, even though I typed `ps` before it finished. The priority 0 `spin` finished much later (102 ticks). This is the starvation problem that Task 5 and the extra credit address.

```
$ spin 40 &
$ pexec 5 spin 40 &
$ ps
spin pid 12 (prio 5) done: 51 ticks
pid     state           size    age     prio    eprio   cputime ppid    name
1       sleeping        16384   0       0       0       1       0       init
2       sleeping        20480   0       0       0       0       1       sh
13      running         16384   0       0       0       0       2       ps
9       runnable        16384   0       0       0       32      1       spin
$ spin pid 9 (prio 0) done: 102 ticks
ps
pid     state           size    age     prio    eprio   cputime ppid    name
1       sleeping        16384   0       0       0       1       0       init
2       sleeping        20480   0       0       0       0       1       sh
14      running         16384   0       0       0       0       2       ps
$ ps
pid     state           size    age     prio    eprio   cputime ppid    name
1       sleeping        16384   0       0       0       1       0       init
2       sleeping        20480   0       0       0       0       1       sh
15      running         16384   0       0       0       0       2       ps
$
```

Figure 7. pexec test on the priority scheduler (AGING=0): the shell is starved

What I learned

I learned that changing the scheduling policy mainly involved changing how `scheduler()` selects the next process. The context switches and lock hand-offs stayed the same. I also learned why the selected process is checked again after taking its lock, even though its state cannot change between the two steps in this single-CPU setup.

The tests showed how strict priority affects lower-priority processes. The highest-priority job ran first, but the shell and `init` could be left waiting while higher-priority jobs used the CPU.

Difficulties

I had to decide how to handle equal priorities. Always choosing the first process in the table would favor the lowest slot, so I used `readytime` to favor the process that had waited longer. Equal-priority processes tend to alternate because `readytime` is reset when they yield, although identical timestamps still favor the first table entry.

Getting repeatable timing results was harder. My first `spin` loop finished in less than a tick, so I adjusted it. Its speed still varied between VM boots, and the common release tick only partly solved the timing problem. I discuss these limits with Table 1.

Without aging, a background job with a higher priority than the shell also made the prompt appear to hang. Figure 7 shows that behavior.

Task 5. (15 pts) Add aging to your priority-based scheduler.

Aging policy

I used the policy suggested in the assignment:

```
effective priority = min(MAXEFFPRIORITY,
    priority + (ticks - readytime) / AGINGTICKS)
```

With `AGINGTICKS = 1`, a process gains one effective-priority point for every tick it waits in the ready queue, up to 99. The scheduler calculates this value in `effpriority()` rather than storing a separate age counter.

The bonus only applies while the process is `RUNNABLE`. When it runs and later becomes `RUNNABLE` again, `readytime` is reset, so it starts waiting from its base priority again. I also added the effective priority to the `ps` output as `eprio`, which helped me check that aging was working. Increasing `AGINGTICKS` would make aging slower.

```
int
effpriority(struct proc *p)
{
  int eff = p->priority;
#if SCHEDPOLICY == SCHED_PRIORITY && AGING
  if (p->state == RUNNABLE) {
    uint waited = ticks - p->readytime;   // the process's "age"
    eff += waited / AGINGTICKS;
  }
#endif
  if (eff > MAXEFFPRIORITY)
    eff = MAXEFFPRIORITY;
  return eff;
}
```

Figure 8. effpriority() with aging in kernel/proc.c

Results

Figure 9 is the same `schedtest` workload as Figure 6, now with aging. The highest priority job still starts first, but the lower priority jobs no longer wait for everyone above them to finish. The priority 30 job gets the CPU at tick 10, once it has aged enough to catch up with priority 40, the priority 20 job at tick 15, and the priority 10 job at tick 20 instead of tick 80. The average response time drops from 40.2 ticks to 11.2 ticks.

```
$ schedtest 20 10 20 30 40
pid     prio    response        turnaround   (ticks, in finishing order)
27      40      0               15
26      30      10              35
25      20      15              55
24      10      20              71
average response   = 11.2 ticks
average turnaround = 44.0 ticks
$
```

Figure 9. schedtest on the priority scheduler with aging

Figure 10 repeats the `pexec` test from Figure 7 with aging on. The screenshot starts with the `schedtest` run from Figure 9, then runs the two background jobs and `ps`. This time `ps` printed right away instead of waiting for the priority 5 job to finish. In the table, the priority 0 `spin` (`pid`

9) has waited 5 ticks (`age` 5), and its effective priority is already 5, the same as the priority 5 `spin` (`eprio` 5). It has caught up and will share the CPU instead of waiting for the other job to finish. Compare this to Figure 7, where `ps` could not run until the priority 5 job was done.

```
$ spin 40 &
pexec 5 spin 40 &
pexec 10 ps$ $
pid     state           size    age     prio    eprio   cputime ppid    name
1       sleeping        16384   0       0       0       0       0       init
2       sleeping        20480   0       0       0       1       1       sh
12      runnable        16384   0       5       5       20      11      spin
9       runnable        16384   5       0       5       4       1       spin
11      sleeping        16384   0       5       5       0       1       pexec
13      sleeping        16384   0       10      10      0       2       pexec
14      running         16384   0       10      10      0       13      ps
$ spin pid 12 (prio 5) done: 54 ticks
spin pid 9 (prio 0) done: 90 ticks

$ QEMU: Terminated
```

Figure 10. pexec test with aging: ps runs immediately

Evaluation: average turnaround and response time

Table 1 compares the three scheduler configurations using `schedtest 20 10 20 30 40`. Each of the four jobs performs the same amount of work. All results are in ticks, and Figure 11 shows the round-robin run.

These were separate runs after booting the VM, and the loop ran at different speeds. The final job finished at tick 56 with round-robin, 106 with priority scheduling, and 71 with aging. Because of that difference, the turnaround times are not directly comparable. Execution speed also affects response time, especially when a lower-priority job must wait for another job to finish.

These are results from individual runs, so they show what happened in those tests rather than values I would expect every time. `schedtest` also truncates averages to one decimal place; for example, 40.25 prints as 40.2.

Test (work = 20 units each)	Round-robin	Priority	Priority + aging
`schedtest 20 10 20 30 40`: average response	1.5	40.2	11.2
`schedtest 20 10 20 30 40`: average turnaround	55.0	66.7	44.0
Priority 40 job: response / turnaround	3 / 56	0 / 27	0 / 15
Priority 10 job: response / turnaround	0 / 55	80 / 106	20 / 71

Table 1. Average response and turnaround time (ticks)

```
init: starting sh
$ schedtest 20 10 20 30 40
pid     prio    response        turnaround   (ticks, in finishing order)
6       30      2               54
4       10      0               55
5       20      1               55
7       40      3               56
average response   = 1.5 ticks
average turnaround = 55.0 ticks
$
```

Figure 11. Baseline: schedtest on the round-robin scheduler

Round-robin had the lowest average response time in these runs: 1.5 ticks, with every job starting within 3 ticks. All four jobs finished around tick 55.

Without aging, the priority-40 job started immediately and finished at tick 27. The priority-10 job waited until tick 80 before it started. With aging, the highest-priority job still started first, but the lower-priority jobs received CPU time sooner. Average response time was 11.2 ticks instead of 40.2, and the priority-10 job started at tick 20 instead of tick 80.

Average turnaround time was also lower in the aging run, at 44.0 ticks compared with 66.7. However, the difference in loop speed means I cannot attribute that improvement entirely to aging. The shell tests in Figures 7 and 10 give a clearer example of the benefit: with aging enabled, the shell could respond while the background jobs were still running.

What I learned

I learned how aging gives a waiting process a chance to run even when other processes have higher base priorities. Using `readytime` also keeps the implementation simple because the scheduler can calculate the waiting time when it needs it.

The aging rate controls how quickly waiting processes catch up. Slower aging behaves more like strict priority, while faster aging gives waiting processes CPU time sooner. I only tested `AGINGTICKS = 1`, so I did not measure different aging rates.

Difficulties

During development, `ps` showed aging bonuses even when I selected round-robin scheduling. I fixed that by applying aging only when `SCHEDPOLICY` is `SCHED_PRIORITY` and `AGING` is enabled.

I also had to run `make clean` when switching configurations. The dependency files track changes to headers, but they do not track changes to the `-D` flags passed through the `Makefile`. Without cleaning, the build could reuse the old kernel.

Task 6. (5 pts) Summary

I learned how the different parts of a kernel change connect: adding fields to `struct proc`, registering system calls, copying process information to user space, and changing how the

scheduler chooses a process. Testing all three configurations helped me see how the scheduling policy affects both background jobs and the shell.

The main problems were sharing `pstat.h` between kernel and user code, fixing the size format in `ps`, and avoiding a deadlock when reading `ticks` during `wakeup()`. Getting consistent timing results was only partly solved. The counts still varied between VM boots, so I treated the measurements as individual examples rather than exact performance comparisons.

The tests showed the behavior I expected from each policy. Round-robin let every job start quickly. Strict priority served higher-priority jobs first but could leave lower-priority processes waiting. Aging gave those waiting processes a chance to run.

Extra Credit Task (10 points).

I implemented `getpriority` and `setpriority` as shell built-ins so they operate on the shell process. A separate `setpriority` program would change its own priority and then exit, leaving the shell unchanged. This is the same reason `cd` runs inside the shell.

In `user/sh.c`, I added the checks after `cd` and before the normal `fork/exec` path. `getpriority` prints the shell's priority, and `setpriority N` changes it. The argument must contain one or two digits, and the syscall checks that its value is between 0 and 49. Checking the text first matters because xv6's `atoi()` converts inputs such as `-1` or `abc` to 0. I added the same argument check to `pexec`.

Commands started afterward inherit the shell's priority unless `pexec` selects a different one. Figure 12 shows the built-ins.

```
  } else if (memcmp(cmd, "getpriority\n", 12) == 0) {
    printf("%d\n", getpriority());
  } else if (memcmp(cmd, "setpriority ", 12) == 0) {
    cmd[strlen(cmd) - 1] = 0; // chop \n
    // atoi() reads "-1" or "abc" as 0, so check the text first.
    char *arg = cmd + 12;
    int ok = (arg[0] != 0), k;
    for (k = 0; arg[k] != 0; k++)
      if (arg[k] < '0' || arg[k] > '9')
        ok = 0;
    if (k > 2)
      ok = 0;
    if (!ok || setpriority(atoi(arg)) < 0)
      fprintf(2, "setpriority: priority must be 0..49\n");
  } else {
```

Figure 12. getpriority and setpriority built-ins in user/sh.c

Figure 13 was run on the priority scheduler with aging turned off, where starvation is worst. `getpriority` printed 0 for the shell. `setpriority 20` changed it, and a second `getpriority` printed 20. `setpriority 60` printed the error message, since 60 is outside 0..49 and the shell keeps its old priority. I then started a `spin` in the background and ran `ps`. The `spin` inherited priority 20 from the shell, and `ps` (also a child of the shell) ran at 20 as well, so it printed while the `spin` was still working (`cputime` 16). With the shell at priority 0, as in Figure 7, `ps` had to wait for a priority 5 job to finish. This is a simple way to keep the shell and interactive commands responsive while long jobs run at lower priority set with `pexec`.

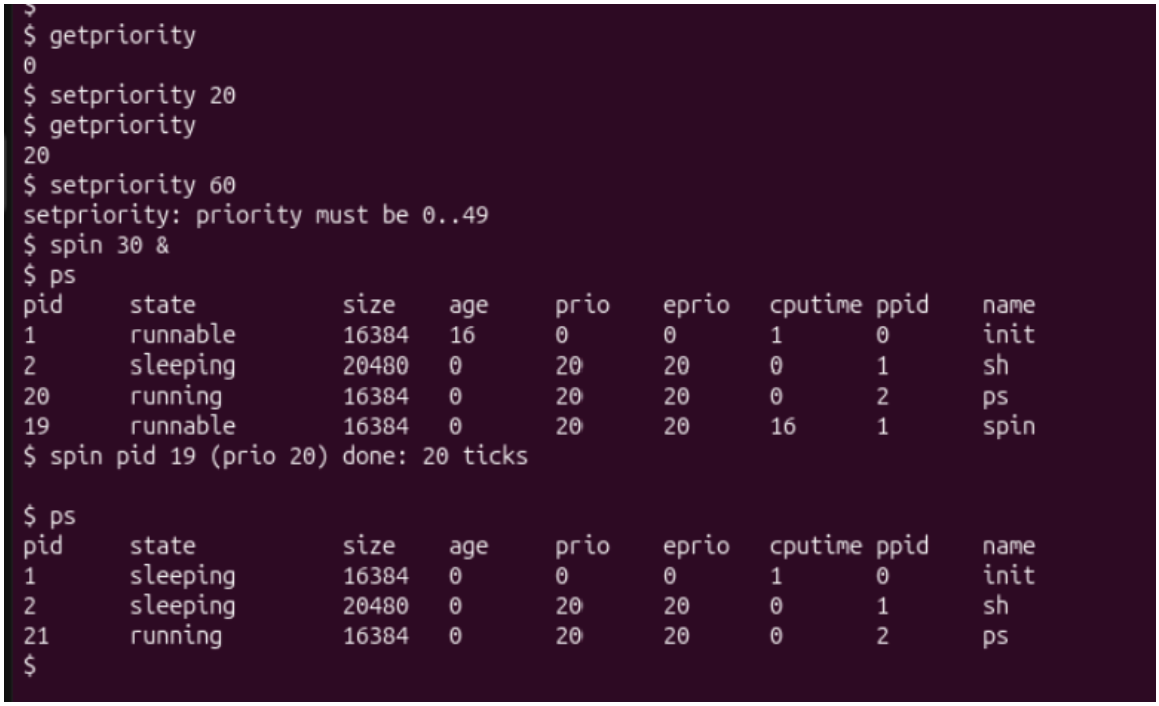


Figure 13. getpriority and setpriority shell built-ins, and ps at shell priority 20

Resources and AI Assistance

Resources used were the xv6-riscv source code, the PA2 assignment handout, and the report template.

I used Claude (Anthropic) to help identify the approach, the files that needed changes, and commands for testing. I used that guidance to understand the direction and scope of the changes. Claude also helped format the report using the provided template.

Claude initially checked the code in its own environment. Every screenshot and numerical result in this report comes from my own runs in my VM, using separate builds for round-robin, priority without aging, and priority with aging.

I also used a separate AI assistant to review the report and repository. Its feedback led to the argument-validation fixes for `setpriority` and `pexec`, along with wording corrections.